TP5 - Analyse d’une application Android vulnérable

Recherche d’informations

brabi@DESKTOP-EH29IVM:~/IoT/tp5$ cat injured_out/resources/AndroidManifest.x

ml | grep -A3 'activity'

```
brabi@DESKTOP-EH29IVM:~/IoT/tp5$ cat injured_out/resources/AndroidManifest.x
ml  | grep -A3 'activity'
        <activity
            android:theme="@style/AppTheme.NoActionBar"
            android:label="@string/title_activity_flag_eighteen"
            android:name="b3nac.injuredandroid.FlagEighteenActivity"
            android:exported="true"/>
        <provider
--
        <activity
            android:theme="@style/AppTheme.NoActionBar"
            android:label="@string/title_activity_flag_seventeen"
            android:name="b3nac.injuredandroid.FlagSeventeenActivity"/>
        <activity
            android:theme="@style/AppTheme.NoActionBar"
            android:label="@string/title_activity_c_s_p_bypass"
            android:name="b3nac.injuredandroid.CSPBypassActivity">
            <intent-filter>
                <action android:name="android.intent.action.VIEW"/>
--
        </activity>
        <activity
            android:theme="@style/AppTheme.NoActionBar"
            android:label="@string/title_activity_assembly"
            android:name="b3nac.injuredandroid.AssemblyActivity"/>
        <activity
            android:theme="@style/AppTheme"
            android:name="io.flutter.embedding.android.FlutterActivity"
            android:configChanges="fontScale|layoutDirection|density|screenS
ize|uiMode|screenLayout|orientation|keyboardHidden|keyboard|locale"
```

- **android:exported="true",** cela signifie que l'activité peut être lancée par n'importe quelle autre application sur le téléphone, ou directement via la ligne de commande avec adb, qui sera exploité après.

Recherche des termes « *key / password / secret / token / flag* » , pour rechercher des informations intéressantes :

grep -rEi 'api_key|password|secret|token|flag' injured_out/sources/

```
brabi@DESKTOP-EH29IVM:~/IoT/tp5$ grep -rEi 'api_key|password|secret|token|fl
ag' injured_out/sources/
injured_out/sources/com/google/firebase/database/b.java:        f2003c.put(-
6, "The supplied auth token has expired");
injured_out/sources/com/google/firebase/database/b.java:        f2003c.put(-
7, "The supplied auth token was invalid");
injured_out/sources/com/google/firebase/database/b.java:        f2004d.put("
expired_token", -6);
injured_out/sources/com/google/firebase/database/b.java:        f2004d.put("
invalid_token", -7);
injured_out/sources/com/google/firebase/database/p/g.java:        com.google
.android.gms.common.internal.r.i(this.f2170c, "You must register an authToke
nProvider before initializing Context.");
injured_out/sources/com/google/firebase/database/p/n.java:            n.this
.j.b("Auth token changed, triggering auth token refresh", new Object[0]);
injured_out/sources/com/google/firebase/database/o/i.java:
  i.this.t.b("Ignoring getToken result, because this was not the latest atte
mpt.", new Object[0]);
```

- On peut y trouver plusieurs informations, par exemple :
  - Flag 1 : le fichier “FlagOneLoginActivity.java” contient : "The flag is right under your nose" et "The flag is also under the GUI".
  - Dans R.java, on voit AWS_SECRET, google_api_key.
  - Flag 7 : le fichier « FlagSevenSqliteActivity.java » parle de flagSevenEncrypted.

Flag 1 :

Analyse du code source du fichier « *FlagOneLoginActivity.java* »

cat injured_out/sources/b3nac/injuredandroid/FlagOneLoginActivity.java

En lisant le code source, on y voit la méthode **submitFlag** qui se trouve à la fin du fichier.

```
public final void submitFlag(View view) {
    EditText editText = (EditText) findViewById(R.id.editText2);
    d.s.d.g.d(editText, "editText2");
    if (d.s.d.g.a(editText.getText().toString(), "F1ag_0n3")) {
        Intent intent = new Intent(this, (Class<?>) FlagOneSuccess.class
```

- Un flag est écrit en « dur », il est lisible à partir du moment on peut décompiler l'APK : **CWE-798 : Use of Hard-coded Credentials**

Flag 2 :

cat injured_out/resources/AndroidManifest.xml : en lisant, on voit qu'une activité b25lActivity est en **exported="true"** et donc accessible de l'extérieur.

```
<activity
    android:name="b3nac.injuredandroid.b25lActivity"
    android:exported="true"/>
<activity
```

cat injured_out/sources/b3nac/injuredandroid/b25lActivity.java : en lisant le code source, on voit que cette activité peut valider un flag dès son ouverture

```
        super.onCreate(bundle);
        setContentView(R.layout.activity_b25l);
        j.j.a(this);
        new FlagsOverview().M(true);
        new j().b(this, "flagTwoButtonColor", true);
    }
```

Lecture du fichier « *activity_b25l.xml* » :

cat injured_out/resources/res/layout/activity_b25l.xml

On y voit un deuxième flag dans **text=«S3c0nd_F1ag»**

```
brabi@DESKTOP-EH29IVM:~/IoT/tp5$ cat injured_out/resources/res/layout/activi
ty_b25l.xml
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://sch
emas.android.com/apk/res/android" xmlns:app="http://schemas.android.com/apk/
res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <TextView
        android:textSize="24sp"
        android:id="@+id/textView6"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="8dp"
        android:layout_marginBottom="8dp"
        android:text="S3c0nd_F1ag"
        android:layout_marginStart="8dp"
        android:layout_marginEnd="8dp"
```

- b25lActivity est accessible, via adb car est exported="true": **CWE-926 : Improper Export of Android Application Components**.

Flag 3

Lecture du fichier strings.xml (souvent critique)

cat injured_out/resources/res/values/strings.xml

On peut y lire un autre flag

```
<string name="clear_text_end_icon_content_description"

<string name="cmVzb3VyY2VzX3lv">F1ag_thr33</string>
<string name="common_google_play_services_enable_butto
<string name="common_google_play_services_enable_text"
```

- Il s'agit d'une fuite d'information via des ressources : **CWE-312 : Cleartext Storage of Sensitive Information**

Flag 4

On va chercher la logique de validation dans le fichier Java « *FlagFourActivity.java* »

cat injured_out/sources/b3nac/injuredandroid/FlagFourActivity.java

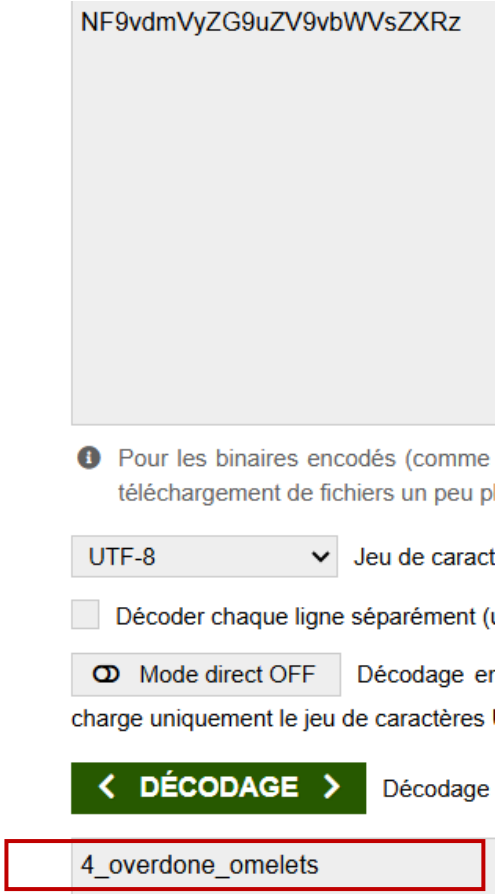
On va ensuite chercher le code de la classe « *g* » (car il y a un appel).

cat injured_out/sources/b3nac/injuredandroid/g.java

On y voit une chaine codée en Base64.

```
brabi@DESKTOP-EH29IVM:~/IoT/tp5$ cat injured_out/sources/b3nac/injuredandroi
d/g.java
package b3nac.injuredandroid;

import android.util.Base64;

/* loaded from: classes.dex */
public class g {

    /* renamed from: a, reason: collision with root package name */
    private byte[] f1468a = Base64.decode("NF9vdmVyZG9uZV9vbWVsZXRz", 0);

    public byte[] a() {
        return this.f1468a;
```

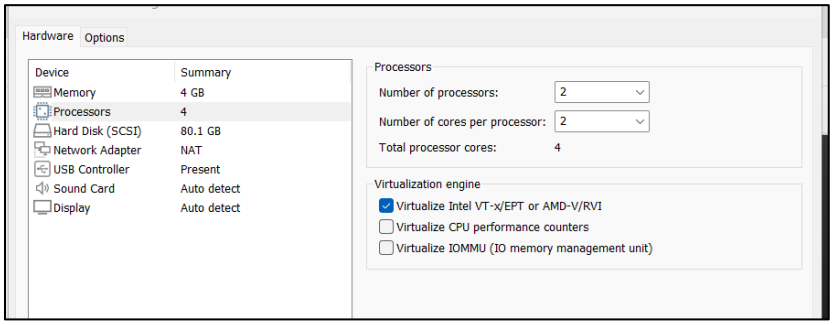
Une fois décodé, on a le 4e flag.

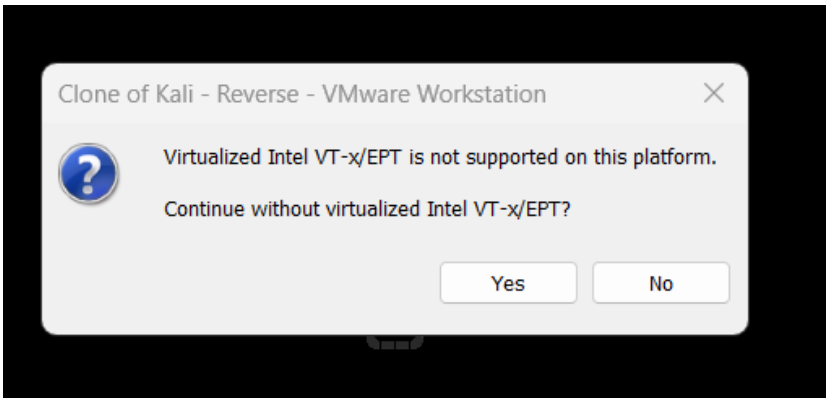


- Il y a obfuscation, mais avec un encodage en base 64, insuffisant en termes de sécurité : **CWE-798 (Hardcoded Credentials)**.

Analyse dynamique avec Frida

Je n'ai pas réussi à faire fonctionner l'émulateur sur ma machine virtuelle Kali, car l'OS ne semble pas compatible avec la virtualisation Intel VT-x/EPT. Je suis donc passé sur Docker.





Docker & SSL Bypass

Suite à des soucis pour lancer, je suis passé par PowerShell.

Le script fourni a permit grâce à l'injection du script fourni, en outrepassant la fonction de vérification.

```
PS Microsoft.PowerShell.Core\FileSystem::\\wsl$\Ubuntu\home\brabi\IoT\tp5
> frida -D emulator-5554 -f b3nac.injuredandroid -l bypass_ssl.js
     ____
    / _  |   Frida 17.9.1 - A world-class dynamic instrumentation toolkit
   | (_| |
    > _  |   Commands:
   /_/ |_|       help      -> Displays the help system
   . . . .       object?   -> Display information about 'object'
   . . . .       exit/quit -> Exit
   . . . .
   . . . .   More info at https://frida.re/docs/home/
   . . . .
   . . . .   Connected to Android Emulator 5554 (id=emulator-5554)
Spawned `b3nac.injuredandroid`. Resuming main thread!
[Android Emulator 5554::b3nac.injuredandroid ]-> ls
ReferenceError: 'ls' is not defined
    at <eval> (<input>:1)
[Android Emulator 5554::b3nac.injuredandroid ]-> Java.available
true
[Android Emulator 5554::b3nac.injuredandroid ]-> Process.id
6148
```

Recommandations à mettre en place

Pour les flags 1 & 3

- Utiliser des outils pour obfusquer le code plus difficile à lire après la décompilation.
- Ne pas stocker de clés API ou données sensibles en clair dans les fichiers strings.xml ou dans le code.

Pour le flag 2

Dans le fichier AndroidManifest.xml, il faudrait changer l'attribut **android:exported="true"** en **android:exported="false"** pour les activités qui n'ont pas besoin d'être appelées par d'autres applications.

Pour le flag 4

Il ne faut pas stocker de secrets dans le code ou l'APK.

Ne pas utiliser de Base64, car c'est un encodage, pas un chiffrement. C'est fait pour transporter des données, pas pour les cacher.

Si vraiment une donnée doit rester dans l'application et être cachée, elle doit être chiffrée (AES-256 par exemple).